
第 9 章

AI Agent 编程基础

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

9. AI Agent 编程基础

本页是**第二部分（AI Agent 编程）**第 9 章的章首页。基线版本：dsh 0.1.5-rc.2 (npm latest, 2026-09-10 发布)；教材核对日期 2026-09-19；维护件见 docs/build/dsh_version.yaml。

本章概览

第一部分教会你用 NumPy、SciPy、Pandas、Matplotlib、NetworkX、Statsmodels、scikit-learn 做科学计算；从本章开始，我们换一个视角：**当 AI 能自己读文件、跑命令、改代码、看报错时，科学计算的工作方式会变成什么样？**

本章以 DeepSeek Harness (dsh) 为例——一个“一切皆插件”的开源 agent harness，讲清三件事：它是什么（概念与循环）、怎么装起来（三条路径 + API Key）、怎么安全地用（权限与数据红线）。

学习目标

1. 说清代码补全、对话式编程与 Agent 编程的区别，描述一次 Agent 循环五个动作；
2. 用“五件套”（模型、工具、上下文、权限、会话）解释一个 harness 的组成；
3. 独立完成 dsh 的三条安装路径之一（npx 免安装 / 全局 CLI / 桌面版源码构建）；
4. 完成 DeepSeek API Key 的注册、实名、充值、创建与填写，并估算一次任务的花费；
5. 会用界面与斜杠命令（/plan、/compact、/goal、/model、/permission、/export 等）控制会话；
6. 说出权限三档、审批机制与数据红线，知道什么数据不能交给云端模型。

先修要求与运行环境

- 先修：第 0 章（终端、Python 环境）与第 1 章 NumPy（本部分案例会用 axis、广播、形状判据）；
- 运行环境：Node.js ^22.19.0 || >= 24.0.0 (npm 与 npx 随它一起安装)、一个 DeepSeek 开放平台账号与 API Key；
- 详细安装步骤见本章 02 节；无 Key 时也能完成理论、审查类练习与离线替代路径。

本章结构

小节	内容	状态
01 AI 编程与 Agent 基础	三代工具、harness 五件套、循环、上下文、人机分工、失败模式	已交付
02 安装与配置	Node.js 与 npm、三条安装路径、API Key 申请与付费、价格快照、故障排查	已交付
03 界面与斜杠命令	Web 界面导览、斜杠命令速查与示例（本地执行 vs 模型对话）	编写中
04 权限、安全与数据红线	权限三档、审批实践、数据红线与合规	编写中

资源导航

资源	位置	说明
理论篇	01-AI 编程与 Agent 基础.md	概念、循环、上下文、权限、失败模式

资源	位置	说明
安装与配置	02-安装与配置.md	三条路径、API Key 与付费、成本估算、故障排查
参考资料	references.md	官方文档、平台入口、通识材料
教学说明	teaching.md	教师用：课时、演示脚本、常见坑
练习	../11-agent-cases/exercises/README.md	全部分练习统一入口（quiz + assignment）
上机	../11-agent-cases/lab/README.md	全部分上机入口（离线可跑 + 联网任务）

版本与时效

本章内容与 dsh 版本强相关，因此有三层防护：

- 1. 版本单一事实源：docs/build/dsh_version.yaml（基线版本、核对日期、上游链接、需复核文件清单）；
- 2. 时效标记：正文中易变段落以“时效内容（基线 vX / 日期）”开头，便于批量复核；
- 3. 双周巡检:docs/build/check_dsh_release.py与.github/workflows/dsh-release-watch.yml在官方发新版时提醒，流程见 docs/build/dsh_更新 SOP.md。

常见问题

问题	处理
没有 API Key 能学吗？	能：本章理论、第 10 章方法、第 11 章全部审查与离线练习都不需要 Key
一定要用 DeepSeek 吗？	教材以 DeepSeek Harness + DeepSeek 模型为主线；dsh 也支持自定义兼容端点
会不会很贵？	练习任务用 deepseek-flash，一次通常几分钱；估算方法见 02 节

01 AI 编程与 Agent 基础

本节回答三个问题：Agent 编程与“AI 帮我写代码”到底差在哪？一个 Agent 内部做了什么？哪些事必须自己管？阅读约 25 分钟；配套图 4 张，可直接运行的代码 1 段。

本节目标

1. 说清**代码补全**、**对话式编程**、**Agent 编程**三者的差别，并解释“闭环”为什么是关键。
2. 用**五件套**描述一个 agent harness（智能体框架）：模型、工具、上下文、权限、会话。
3. 复述一次 **Agent 循环** 的五个动作，并能把一个小任务拆成循环。
4. 解释**上下文窗口与压缩**，知道怎么“省上下文”。
5. 说出**权限三档**与审批机制，记住数据红线在哪里。
6. 识别**四种典型失败模式**，并给出至少一条对策。

先修

- 完成第 0 章的环境安装：会用终端（PowerShell / Bash）、会运行 python 脚本；
- 学完第 1 章 NumPy：本节的演示案例用到 axis 与广播；
- 不需要任何机器学习或大模型背景。

术语速查（中英对照）

后续章节会反复用到这些词，先混个脸熟即可：

术语	英文	一句话解释
智能体	Agent	能自己调用工具、执行多步任务来完成目标的 AI 程序
智能体框架	Agent harness	把模型、工具、上下文、权限、会话组装起来跑循环的软件
工具调用	Tool call	模型请求执行一个动作（读文件、跑命令、搜索）并拿到结果
上下文	Context	模型这一次能看到的全部内容：指令、历史、文件片段、工具输出
压缩	Compaction	上下文快满时，把过程压缩成“目标 + 结论 + 证据”
权限	Permission	决定 Agent 能做什么、哪些动作需要你点头
沙箱	Sandbox	限制 Agent 只能访问指定范围的机制
技能	Skill	预先写好的领域工作流，按需加载进上下文
子代理	Subagent	由主代理派生、专门做一件事的独立 Agent
计划模式	Plan mode	先产出计划、经你确认后再动手的工作方式
会话	Session	一次持续的对话上下文，可保存、继续、回看
工作区	Workspace	Agent 默认能读写的项目目录

正文

1. 三代编程助手：从“猜下一行”到“自己动手”



Figure 1: 三代编程助手对比

	代码补全	对话式编程	Agent 编程
你给它	正在写的这一行	报错、代码片段、问题	目标 + 约束 + 验收标准
它给你	猜出的下一行	代码与解释	读文件、跑命令、改代码、再验证的 完整过程
你负责	想清楚、跑通、验证	复制粘贴、串起来、验证	定义问题、定验收、做终审
出错时	你改	你改	它自己看报错再改（直到你设的验收标准满足）

三者的分水岭是**闭环**：前两代的输出是“文本”，你拿到文本后还要自己去运行、观察、判断、再回来描述问题；Agent 的输出是“完成的动作”——它有眼睛（读文件、看报错）和手（改文件、跑命令），能在这个循环里自我修正。

一句话记住：**补全帮你写一行，对话帮你写一段，Agent 帮你把事情做完——但“做对”的定义仍然由你给出。**

2. harness：模型之外的四件事

很多人以为“用了同一个模型，效果应该差不多”。实际上，同一个模型在不同 harness 里的表现可以差很远，因为决定体验的是另外四件事：



Figure 2: harness 的组成

组成	作用	少了它会怎样
模型 LLM	推理与决策	没有大脑，什么也做不了
工具 Tools	读写文件、跑命令、搜索	只能说不能做，等于聊天机器人
上下文 Context	把项目指令、历史、文件内容喂给模型	它不知道你的项目规范，反复问同样的问题
权限 Permission	决定能不能写、要不要审批	要么什么都不敢做，要么风险失控
会话 Session	记录全部动作，可继续、可回看	每次从零开始，出了问题无法追责

类比：模型是发动机，harness 是底盘、变速箱、刹车和仪表盘。发动机再好，没有刹车也不敢上路。

3. 一次 Agent 循环的解剖



2. **想**：拆解任务、定计划。复杂任务建议先进入**计划模式**，让你确认后再动手。
3. **做**：调用工具——改代码、跑脚本、装依赖、查文档。
4. **看**：读回执行结果。这一步是 Agent 与“聊天”最大的区别：它必须面对真实输出。
5. **判**：达标了吗？达标的判据**不是**“代码跑起来了”，而是你事先给的**验收标准**。

用一个具体任务走一遍。假设你有一份成绩矩阵，想算“每个学生的平均分”，但代码写成了按列求平均：

- 读：Agent 打开脚本，看到 scores 的形状是 (4, 3)、代码里写着 mean(axis=0)。
- 想：它需要判断你的**意图**——“每个学生”对应行还是列。
- 做：它写出最小复现，分别打印两种 axis 的结果和形状。
- 看：形状一个 (3,) 一个 (4,)，它意识到意图与实现不一致。
- 判：按你给的验收标准（形状必须是 (4,)、数值与手算一致）给出结论与修复。

关键认识：**循环的终止条件由你定义**。没有验收标准，Agent 就会以“看起来能跑”作为终点。

4. 上下文：Agent 的短期记忆

上下文窗口：Agent 的短期记忆

窗口再大也会满；满了就要压缩或遗忘



Figure 4: 上下文窗口与压缩

上下文窗口再大也会满。一次长任务里，系统提示、对话历史、读入的文件、工具输出会迅速堆积；快满时 harness 会做**压缩 (compaction)**，把过程浓缩成目标、结论与关键证据。

三条实用建议：

1. **给最小可复现**：一段 20 行的复现脚本，比 2000 行项目源码更有用。
2. **让它自己读文件**：说“读 src/clean.py 里 load_data 函数”比把代码整段贴进对话更省上下文，也更准确。
3. **长任务主动收口**：阶段结束时让它总结“已确认的结论 + 未解决的问题 + 下一步”，这既是压缩，也是你检查方向的时机。

案例卡 1：一个“看起来都对”的错误——axis 用反了 目标：认识“静默数值错误”——代码能跑、数字看着合理，但结论是错的。

代码

```
import numpy as np
np.set_printoptions(precision=2, suppress=True)

scores = np.array([[82, 90, 78],
                   [60, 65, 70],
                   [95, 88, 92],
                   [70, 72, 68]], dtype=float)

by_exam = scores.mean(axis=0)
by_student = scores.mean(axis=1)

print("by_exam  :", by_exam, "shape", by_exam.shape)
print("by_student:", by_student, "shape", by_student.shape)
print("mean of all:", round(float(scores.mean()), 3))
print("sum check :", round(float(by_exam.sum()), 3), round(float(by_student.sum()), 3))
```

运行结果（已运行核验）

```
by_exam  : [76.75 78.75 77.  ] shape (3,)
by_student: [83.33 65.   91.67 70.  ] shape (4,)
mean of all: 77.5
sum check : 232.5 310.0
```

讲解

- 两个结果都是“合理的分数”，粗看都像平均分——这正是静默错误可怕的地方；
- 形状是第一个判据：想算“每个学生”就该得到 4 个数，得到 3 个说明方向错了；
- 总和是第二个判据：by_exam 的均值之和 232.5 只是总分 930 的三分之一，by_student 的 310.0 才是“每人平均分之和”；
- 只打印数字、不做判据检查，这类错误可以躲过一整天。

主要用法 / API

用法	说明
array.mean(axis=k)	沿第 k 维求平均：k=0 消掉行、k=1 消掉列
array.shape	判断结果方向是否符合意图的第一判据
np.set_printoptions	控制打印精度，避免误以为“数不一样”

常见错误

1. 只把结果打印出来看一眼“像不像”，不用形状与守恒量检查；
2. 把 axis=0 记成“对每一行”（其实是沿行方向压缩，得到每列的值）；
3. 让 Agent 修 bug 时只说“结果不对”，不说期望的形状与判据，它只能猜。

拓展

- 把 scores 换成 (3, 4, 5) 的三维数组，先说出 mean(axis=1) 的形状，再运行验证；
- 让 Agent 写一个 3 行的断言函数，把“形状必须是 (4,)”固化成测试；
- 思考：如果这份数据是某次实验的唯一结果，这个错误会怎样影响结论？

5. 人机分工：什么交给它，什么必须自己管

任务类型	建议	原因
样板代码、格式转换、批量重命名	交给 Agent	规则明确、易验证
写测试、补边界条件、重构	交给 Agent（你审 diff）	有客观判据
查文档、读陌生代码库、写注释	交给 Agent	信息检索型工作
问题定义与建模	人主导	决定“做什么”，Agent 无法替你决定
验收标准与数据判据	人主导	决定“什么叫做对”
结论与解释	人主导 （Agent 只能起草）	涉及因果、局限与责任

三条铁律：**问题定义归人、验收标准归人、结论归人**。Agent 可以在这三件事上给建议，但不能替你签字。

6. 权限与安全：让 Agent 在笼子里干活



Figure 5: 权限档位与审批

主流 harness 都提供分档权限，常见三档：

档位	含义	适用场景
read-only	只能读，不能改	第一次接触陌生项目、只想让它解释代码
workspace-write	只能改工作区内的文件	日常开发、课程作业（ 推荐默认 ）
danger-full-access	全盘可写、命令直通	受控环境下的高级用法，慎用

配套机制是**审批**：涉及写文件、跑命令、装依赖时，界面会先问你；你的每一次允许或拒绝都会写进会话记录，可回看、可追责。

数据红线（请直接记住）：

- 不要把隐私数据（身份证、手机号、成绩单、医疗记录）贴进对话；
- 不要把涉密或未发表的数据交给云端模型；
- 不要把别人的 API Key、密码、令牌放进提示词或代码；
- 处理真实数据前，先想清楚“这份数据能不能离开这台机器”。

7. 四种典型失败模式与对策

失败模式	长什么样	对策
幻觉 API	编出不存在的函数、参数、配置项	要求它运行验证；自己查一眼官方文档
静默数值错误	能跑、数字合理，其实方向或单位错了	事先给判据：形状、量纲、守恒量、解析解对照
过度自信的结论	把相关说成因果、不给不确定性	要求给证据、给区间、给反例，结论自己写
改坏别处	顺手重构、动了不该动的文件	小步提交、看清 diff、跑测试再接受

这四种失败都不是“模型不够聪明”，而是**缺验证**。第 03 节会给出一个固定的验收 workflow。

8. 上手体验（5 分钟）

如果你已经按第 02 节装好了 DeepSeek Harness，现在就可以试一次。任务：让它解释本节的 axis 案例。

我在做课程第 1 章的练习题，这段代码想算“每个学生的平均分”，但我不确定对不对：

```
scores = np.array([[82, 90, 78], [60, 65, 70], [95, 88, 92], [70, 72, 68]], dtype=float)
by_exam = scores.mean(axis=0)
```

请你：

- 1) 指出这段代码算出来的到底是什么（给出形状）；
 - 2) 写出正确写法，并给出一个最小复现脚本；
 - 3) 加一条断言，保证结果形状符合“每个学生”的意图。
- 不要修改我工作区里的其他文件。

观察三个点：

1. 它有没有**先读文件、先运行**，还是直接给答案；
2. 它给出的断言是不是可执行、可验证；
3. 它有没有解释“为什么 axis=0 得到的是每列”，而不是只给结论。

本节小结

1. 三代工具的分水岭是**闭环**：能否自己执行、观察、修正。
2. harness = 模型 + 工具 + 上下文 + 权限 + 会话，后四件决定体验。
3. 循环五步：读、想、做、看、判；**终止条件由你的验收标准定义**。
4. 上下文是最稀缺资源：给最小复现、让它自己读文件、长任务主动收口。
5. 权限三档 + 审批 + 数据红线；问题定义、验收标准、结论永远归人。

思考题

1. 同一段代码，让“对话式助手”和“Agent”分别修一个 bug，你能观察到哪些行为差异？
2. 为什么“代码能跑通”不能作为验收标准？请举一个本课程里的例子。
3. 如果上下文快满了，你会保留哪三类信息？
4. 什么情况下应该主动把权限降到 read-only？
5. 同学说“Agent 写的报告结论和我们实验一致，所以没问题”。请指出这句话的逻辑漏洞。

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 1-4 题（概念题）；
- 延伸：DeepSeek Harness 官方仓库与文档（见 references.md）；
- 下一节：02-安装与配置.md（三条安装路径 + API Key 申请与付费）。

02 安装与配置

时效内容（基线 v0.1.5-rc.2 / 2026-09-19）： 本节涉及版本号、命令、界面与价格，是全书最容易过期的部分。遇到不一致时以官方文档为准；更新流程见 docs/build/dsh_更新 SOP.md。

本节目标

- 1. 按自己的场景选对安装路径：**免安装 Web 版 / 全局 CLI 版 / 桌面版**。
- 2. 独立完成 **API Key** 的注册、实名、充值、创建与填入。
- 3. 配置模型并跑通**第一个任务**，知道怎么验收。
- 4. 说清 dsh 的**版本状态**，会自查版本与升级。
- 5. 会用价格表**估算一次任务的花费**。

先修

- 第 0 章的终端使用与 Python 环境；
- 会看命令行输出、会复制粘贴命令即可，不需要前端或运维经验。

0. 安装前的准备（先读 2 分钟）

0.1 dsh 的版本状态与时效

DeepSeek Harness (dsh) 目前处于官方所说的 **开发者预览 (Developer Preview)** 阶段，迭代很快：

- **教材基线**：0.1.5-rc.2 (npm 的 latest 标签，2026-09-10 发布；教材核对日期 2026-09-19)；
- **官方最新**：撰写时 GitHub Releases 上所有 tag 都标记为 prerelease，没有传统意义的“稳定版”；
- 也就是说：**npm latest 就是官方当前推荐的安装版本**，名字里的 rc 不代表“不能用于教学”。

三条自查命令（建议收藏）：

```
dsh --version                # 本机已装版本
npm view @deepseek-ai/dsh dist-tags    # 官方各渠道最新版本 (Latest / next / alpha)
npm view @deepseek-ai/dsh version      # Latest 渠道的具体版本号
```

官方入口（完整清单见 references.md）：

- 仓库：<https://github.com/deepseek-ai/deepseek-harness>
- 文档站：<https://deepseek-harness.github.io/deepseek-harness/>
- 版本记录：仓库 Releases 页（每个版本都有中文更新说明）

0.2 Node.js、npm、npx 是什么

dsh 本身是用 JavaScript / TypeScript 写的，所以需要有一个 JavaScript 运行时；它的安装包发布在 npm 仓库里，所以还需要一个包管理器。三者关系一句话：

Node.js 提供运行环境 → npm 负责从仓库取包、装包 → dsh 是 npm 上的一个包。

名词	全称	作用	本节里第一次用到
Node.js	——	让 JavaScript 脱离浏览器运行的运行时	运行 dsh
npm	Node Package Manager	Node 的官方包管理器：装包、升级、卸载	npm i -g @deepseek-ai/dsh

名词	全称	作用	本节里第一次用到
npx	npm exec	npm 自带的“临时执行器”：不永久安装，直接下载并运行	npx @deepseek-ai/dsh web

关键点：**安装 Node.js 时会自动装上 npm 与 npx**，不需要（也不应该）单独去装 npm。网上“先装 node 再装 npm”的教程是过时说法。

0.3 安装 Node.js（三平台 + 两种风格）

版本要求：^22.19.0 或 >= 24.0.0。推荐装 **Node 24 LTS**（当前 LTS 代号 Krypton, 24.x）；**Node 22 LTS**（Jod, 22.19 以上）也满足要求。

平台	方式	步骤 / 命令	适合谁
Windows	官网安装包（推荐）	打开 nodejs.org → 下载 LTS 版.msi → 双击安装（保持默认勾选 Add to PATH）→ 重开终端	绝大多数同学
Windows	winget	winget install OpenJS.NodeJS.LTS	已用 winget 管理软件的同学
Windows / macOS	版本管理器 fnm	winget install Schniz.fnm 或 brew install fnm, 再执行 fnm install 24 与 fnm use 24	需要多个 Node 版本共存
macOS	Homebrew	brew install node@24	已装 Homebrew
macOS / Linux	nvm	按 nvm 官方 README 提供的脚本安装，然后 nvm install --lts	服务器、多版本场景
Linux	发行版包管理器 / NodeSource	apt、dnf 等； 注意仓库里的版本可能偏旧 ，装完务必按 0.4 自查	有 root 权限的服务器

两条经验：

- 1. 装完**一定要重开终端**（环境变量 PATH 才会生效），否则会一直提示找不到命令；
- 2. 有版本管理器时优先用它——将来 dsh 要求更高版本时，一条命令就能切换。

0.4 装完自查

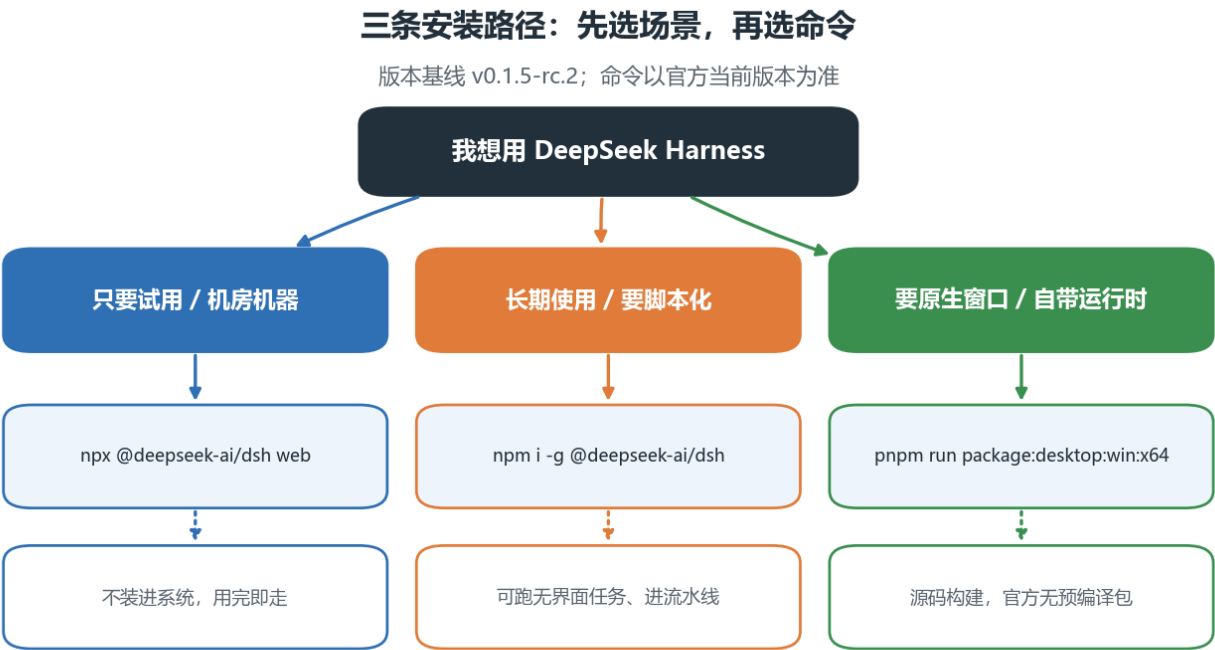
```
node -v           # 期望 v24.x 或 v22.19 以上
npm -v           # 能打印版本号即可
npx --version     # npx 随 npm 一起安装
npm config get registry # 默认 https://registry.npmjs.org/
```

如果 npm config get registry 显示的是公司内网地址，说明这台机器配置了私有仓库，安装 dsh 前请先确认该仓库能拉到公开包（否则改用个人电脑，或按管理员要求配置代理）。

0.5 装不上怎么办

现象	可能原因	处理
node 不是内部或外部命令 / command not found	没装成功，或 PATH 未生效	重开终端；仍不行则重装（Windows 安装时保持勾选 Add to PATH）
npm 报 EACCES / EPERM 权限错误	用管理员或 root 装了全局包	不要用 sudo / 管理员装全局包；用普通用户重装 Node 后再试
下载很慢或超时	网络问题	临时换镜像：npm config set registry https://registry.npmirror.com; 或按校网要求配置代理
版本太低（例如 v20）	系统包管理器里的版本旧	用 fnm / nvm 安装 24 LTS，或从官网重装
公司电脑装不了	安装策略限制	用官网 zip 免安装版解压后手动加 PATH，或改用学校机器
卸载后旧版本还在	多个 Node 共存	Windows 用 where node、macOS / Linux 用 which -a node 查看来源并清理

1. 三条路径怎么选



共同前置：先装 Node.js（自带 npm 与 npx），要求 ^22.19.0 或 >= 24

Figure 1: 三条安装路径

路径	适合谁	需要准备	升级方式
A 免安装 Web 版	第一次用、机房机器、不想改系统	Node.js	每次运行自动取最新（也可锁定版本）
B 全局 CLI 版	长期使用、要写脚本、要无界面跑任务	Node.js + npm	npm i -g @deepseek-ai/dsh@latest
C 桌面版	想要原生窗口与自带运行时	Node.js + pnpm + 源码构建	重新拉源码构建

共同前置只有一个：**Node.js**（npm 与 npx 随它一起安装，见 0.2 与 0.3）。官方仓库要求的版本是 ^22.19.0 或 >= 24.0.0。

```
node -v      # 例如 v24.19.0
npm -v       # 例如 11.x
```

如果 Node 版本过低，先升级：官网下载 LTS 版，或用 fnm / nvm 等版本管理器（第 0 章介绍过管理多个工具链的思路）。

提示：npm 下载慢时，可以临时用国内镜像(npm config set registry https://registry.npmmirror.com); 企业网络或校园网如有代理，请按网络管理员要求配置。

2. 路径 A：免安装 Web 版（推荐先试）

不安装任何全局命令，直接用 npx 拉起浏览器界面：

```
npx @deepseek-ai/dsh web
```

会发生什么：

- 1. 首次运行会下载依赖包（视网络情况可能需要几分钟），请耐心等待；
- 2. 服务默认在 http://127.0.0.1:3080 启动，本机运行时会自动打开浏览器；
- 3. **启动命令时所在的目录，就是默认工作区**，建议在一个专门的项目目录里执行；
- 4. 关闭：回到终端按 Ctrl+C。

常用参数：

```
npx @deepseek-ai/dsh web --port 8080      # 换端口（被占用时很有用）
npx @deepseek-ai/dsh web --no-open        # 只启动服务，不开浏览器
npx @deepseek-ai/dsh web --help          # 查看当前版本支持的全部参数
```

说明：如果通过 SSH 在服务器上启动，终端只会打印地址，需要你本地做端口转发（进阶用法，本节不展开）。

3. 路径 B：全局 CLI 版（长期使用）

装一次，长期用：

```
npm i -g @deepseek-ai/dsh      # 全局安装
dsh --version                  # 确认版本
dsh web                        # 等价于 npx 的浏览器界面
```

本地版真正多出来的能力是**无界面（headless）任务**——让 Agent 跑完一件事直接打印结果，适合写进脚本：

```
dsh --profile headless " 统计当前目录下所有 Python 文件的行数，输出一个表格"
```

其他自查入口：

```
dsh --help           # 启动器自身的参数
dsh web --help       # Web 应用的参数
dsh --dump-config    # 查看合并后的配置树（不启动）
```

首次运行会在用户目录下创建配置与数据目录（Windows 为 C:/Users/你的用户名/.dsh，macOS 与 Linux 为 ~/.dsh），里面包括设置、会话记录与凭据文件。**凭据文件不要提交到 git，也不要截图外发。**

升级与卸载：

```
npm i -g @deepseek-ai/dsh@latest    # 升级到最新
npm rm -g @deepseek-ai/dsh          # 卸载
```

对比项	npx 免安装	全局 CLI	headless
是否改系统	否	是（全局包）	是
界面	浏览器	浏览器	无（打印结果）
适合	试用、机房	日常使用	批处理、自动化
升级	每次运行自动解析	手动升级	跟随全局

4. 路径 C：桌面版（进阶，官方暂无预编译安装包）

先说结论：**官方目前没有发布桌面端安装包**，桌面端需要从源码构建，工程量偏大。课堂上了解它的定位与取舍即可，不要求真的构建。

桌面端是什么：它是完整 Web 应用外面的一层 Electron 壳——同一套界面与能力，多了原生窗口、菜单与系统通知，并**自带 Python / Node.js / pnpm 运行时**（对不想自己配环境的用户友好）。它默认使用 19387 端口，与 Web 版的 3080 分开。

源码构建（需要 Node 与 pnpm，命令以仓库当前版本为准）：

```
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install
pnpm run build
pnpm run package:desktop:win:x64           # Windows
# macOS Apple 芯片: pnpm run package:desktop:mac:arm64
# macOS Intel: pnpm run package:desktop:mac:x64
```

只想先看看界面，可以用开发模式启动：pnpm run dev:desktop。

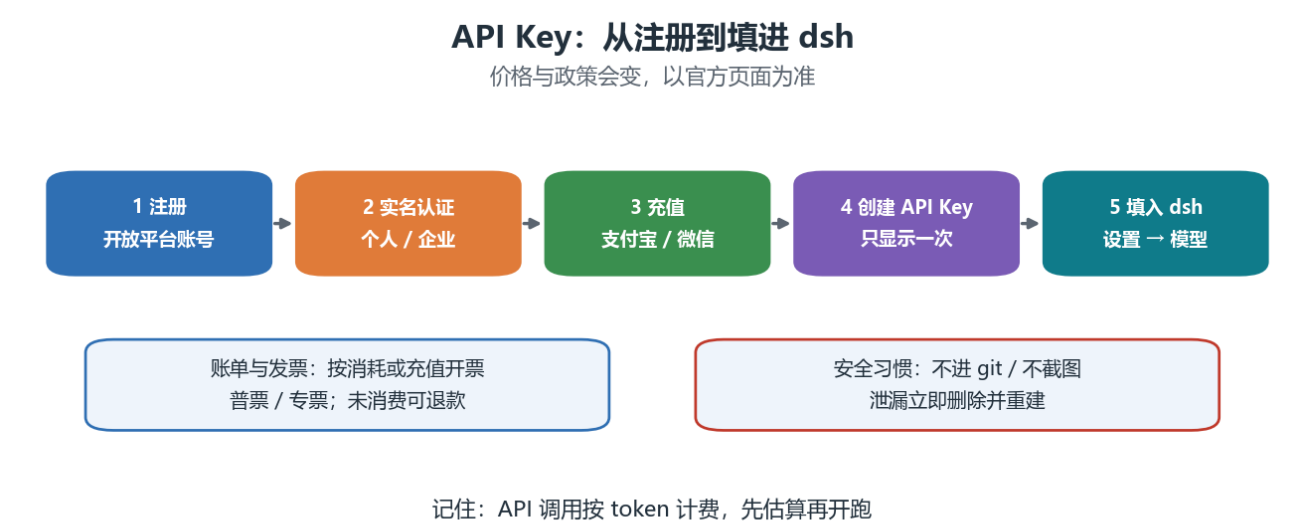
社区项目：有第三方开发者为 Windows 打包了桌面版分发（例如 GitHub 上的 DeepSeek-Harness-Desktop）。它**不是官方发布**，无法审计构建过程，安装前请自行评估供应链风险；课程不把它作为推荐路径。

5. 配置模型与 API Key（重点）

5.1 为什么需要 API Key

dsh 本身是免费开源软件，但它在运行时需要调用大模型，而模型调用按用量计费。因此你需要一个 DeepSeek 开放平台账号与 API Key——可以理解为“给 Agent 用的加油卡”。

5.2 五步拿到 API Key



高峰时段：北京时间周一至周五 9:00–12:00、14:00–18:00；其余时间为空闲时段（半价）。价格与政策会调整，**以官方定价页为准**。课程建议：练习与作业用 deepseek-flash；把重跑安排在空闲时段，能省一半。

5.4 估算一次任务花多少钱

下面这个脚本用一组“典型参数”估算费用：20K 未命中输入 + 60K 命中输入 + 15K 输出。

```
price = {"flash": {"in_miss": 1.0, "in_hit": 0.02, "out": 4.0},
        "v4-pro": {"in_miss": 4.5, "in_hit": 0.15, "out": 13.5}}

def cost(model, in_miss_k, in_hit_k, out_k, peak=False):
    p = price[model]
    factor = 2.0 if peak else 1.0
    return factor * (p["in_miss"] * in_miss_k + p["in_hit"] * in_hit_k + p["out"] * out_k)

print("flash off-peak:", round(cost("flash", 20, 60, 15), 4), "CNY")
print("flash peak      :", round(cost("flash", 20, 60, 15, peak=True), 4), "CNY")
print("v4-pro off-peak:", round(cost("v4-pro", 20, 60, 15), 4), "CNY")

flash off-peak: 0.0812 CNY
flash peak      : 0.1624 CNY
v4-pro off-peak: 0.3015 CNY
```

结论：一次“读代码 + 改代码 + 跑测试 + 写说明”的中等任务，用 deepseek-flash 大约几分钱到几角钱；换成 deepseek-v4-pro 会贵 3 到 4 倍。

6. 第一个任务（5 分钟验收）

- 1. 在浏览器界面点「选择工作区」，添加你的课程项目目录；
- 2. 新建会话，把下面这段提示词发给它：

请阅读当前工作区，回答三个问题：

- 1) 这个项目是做什么的？列出主要目录及用途；
- 2) 有哪些 Python 脚本？各自的作用是什么？
- 3) 如果我要新增一个数据分析脚本，应该放在哪里、遵循什么命名习惯？

只读不改：不要修改任何文件。

- 3. 观察它调用了哪些工具（读文件、列目录、搜索），以及它给出的结论有没有**证据**（引用了具体文件与行）。

验收清单：

- 它确实读了文件，而不是凭空描述；
- 它引用的文件名与目录结构能在你的项目里对上；
- 整个过程没有修改任何文件（因为你在提示词里要求了只读）。

7. 常见故障排查

现象	可能原因	处理
npm: command not found	没装 Node.js	安装 Node LTS 后重开终端

现象	可能原因	处理
npx 卡住不动	网络慢或代理	换网络、配置镜像或代理；耐心等待首次下载
端口 3080 被占用	已有实例在跑	用-port 8080 换端口，或先关闭旧实例
浏览器打不开页面	服务没起来或防火墙拦截	看终端是否打印了地址、有没有报错
输入框灰色不可用	没有选择工作区	先「选择工作区」并选中
模型报鉴权错误	Key 填错或已被删除	重新在「设置 → 模型」粘贴 Key
dsh 全局命令找不到	全局 bin 目录不在 PATH	重开终端；或改回用 npx
PowerShell 执行策略报错	脚本被策略拦截	改用 npx 直接运行，或按学校要求调整策略
任务跑很久没结果	任务过大或网络慢	缩小任务范围，先给最小复现

8. 升级与版本自查

```
npm view @deepseek-ai/dsh dist-tags      # 看官方最新
dsh --version                             # 看本机版本
npm i -g @deepseek-ai/dsh@latest         # 升级
```

升级前建议读一眼 Releases 页的更新说明（官方每个版本都有中文条目），重点关注三类变化：**命令与参数**、**界面位置**、**模型名称**。这三类正是本节内容的来源。

本部分的维护机制:docs/build/dsh_version.yaml 记录基线版本,docs/build/check_dsh_release.py 定期比对方版本，双周自动提醒（见 docs/build/dsh_更新 SOP.md）。

本节小结

- 1. 三条路径：npx 免安装（先试）、全局 CLI（长期与脚本化）、桌面版（源码构建，进阶）。
- 2. 共同前置是 Node.js（^22.19.0 或 >= 24.0.0）。
- 3. API Key 五步：注册、实名、充值、创建、填入「设置 → 模型」。
- 4. 计费按 token、分高峰与空闲；练习用 deepseek-flash，重跑放空闲时段。
- 5. Key 与凭据文件是资产：不进 git、不截图、泄漏立即删除重建。

思考题

- 1. 同学说“我用 npx 跑起来了，所以不用装 Node”——这句话哪里不对？
- 2. 为什么教材不推荐学生用第三方打包的桌面版？请列出两条风险。
- 3. 同样的任务，在高峰时段与空闲时段跑，费用差多少？请用 5.4 的脚本算一个例子。
- 4. 如果 API Key 不小心提交到了 git 仓库，你应该按什么顺序处理？
- 5. 你想让 Agent 每天晚上自动跑一次数据检查，应该选哪条路径、用什么入口？

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 5–6 题；
- 上机：lab/lab.ipynb Part A（环境自检，可无网络完成）；
- 参考：官方 README、Web UI 指南、模型配置指南（见 references.md）。